

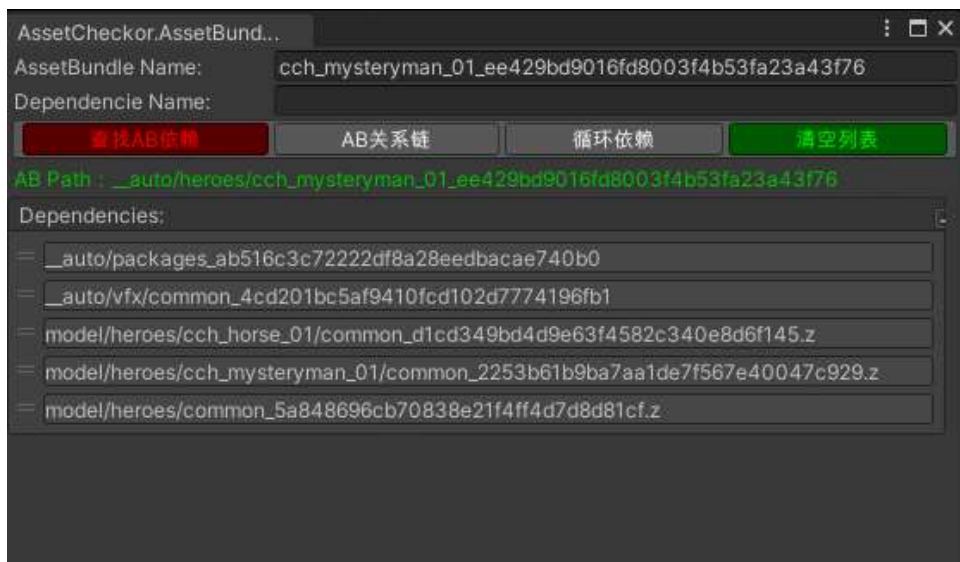
AssetBundleChecker说明文档

Tools->AssetChecker->AssetBundleCheckerWindow

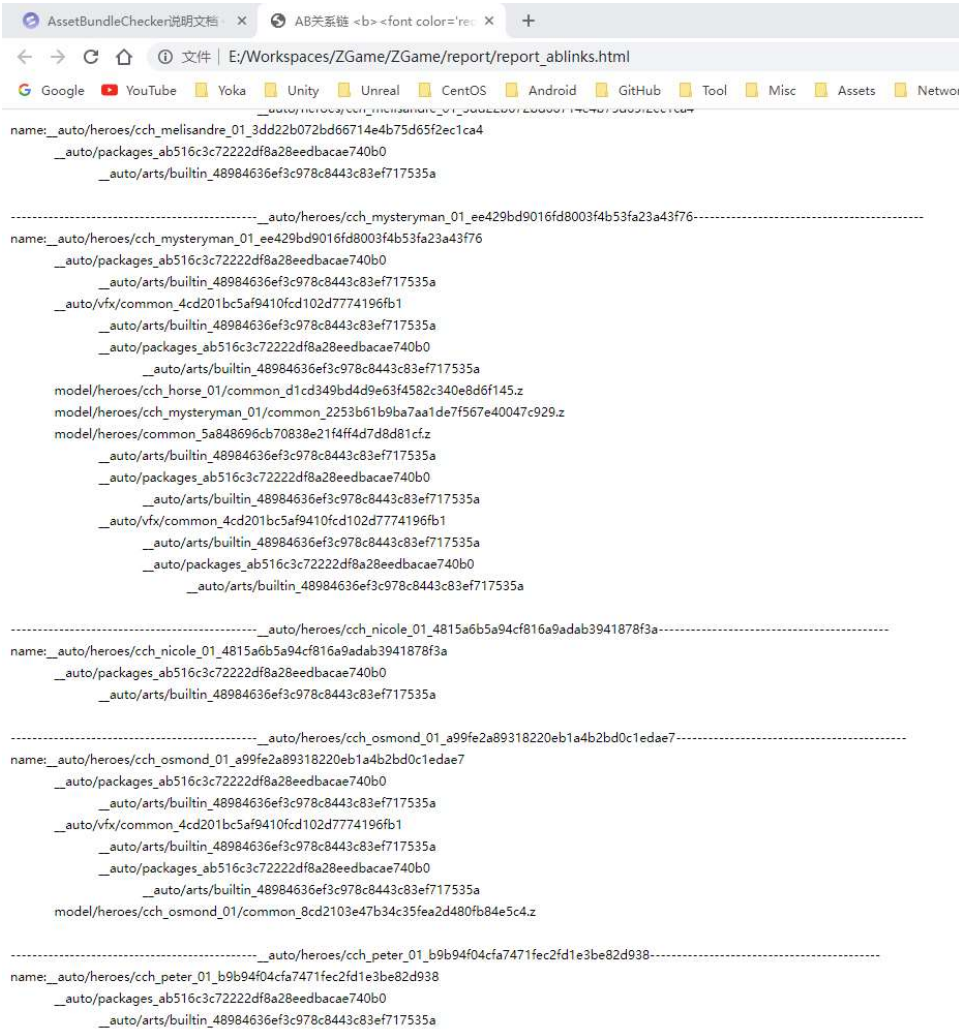
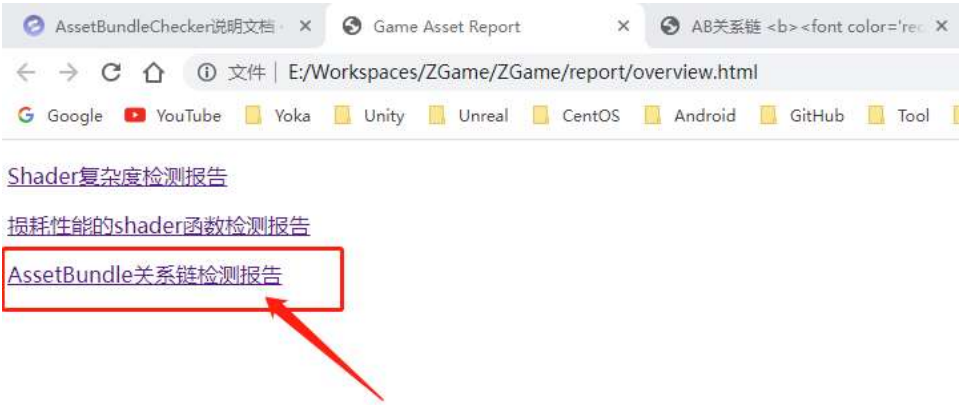
AssetBundleChecker依赖于AssetBundleManifest文件，如果StreamingAssets文件下面没有存在ABManifest文件，以下功能都不能使用，所以需要打AB或者从其他人那里复制这个文件到StreamingAssets目录下。这个功能屏蔽了白名单的部分ab文件，比如shader_xxxx。查找支持模糊查找，可以填入部分名称，不必须是全路径。

AssetBundleChecker分为以下几个子功能：

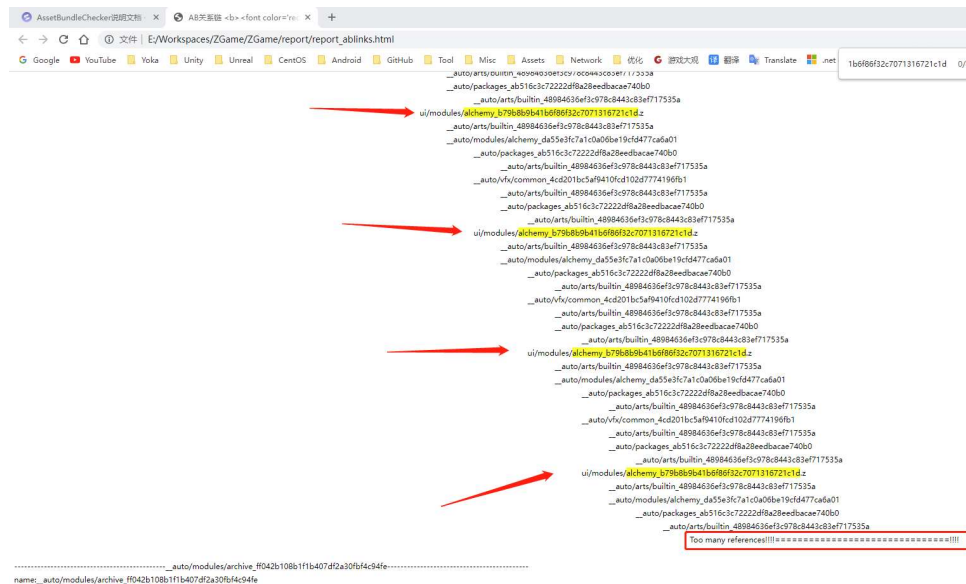
(1) 查找AB依赖功能，功能只能查找第一层的依赖AB，如果需要看完整的AB依赖关系链，需要使用到下个功能。DependencieName功能是可以确认某一个ab是否在当前依赖列表中。如果不填的话，就显示所有的依赖AB列表。



(2) AB关系链功能，这个功能会在report报告目录下生成AssetBundle关系链检测报告，可以通过下面链接进入。



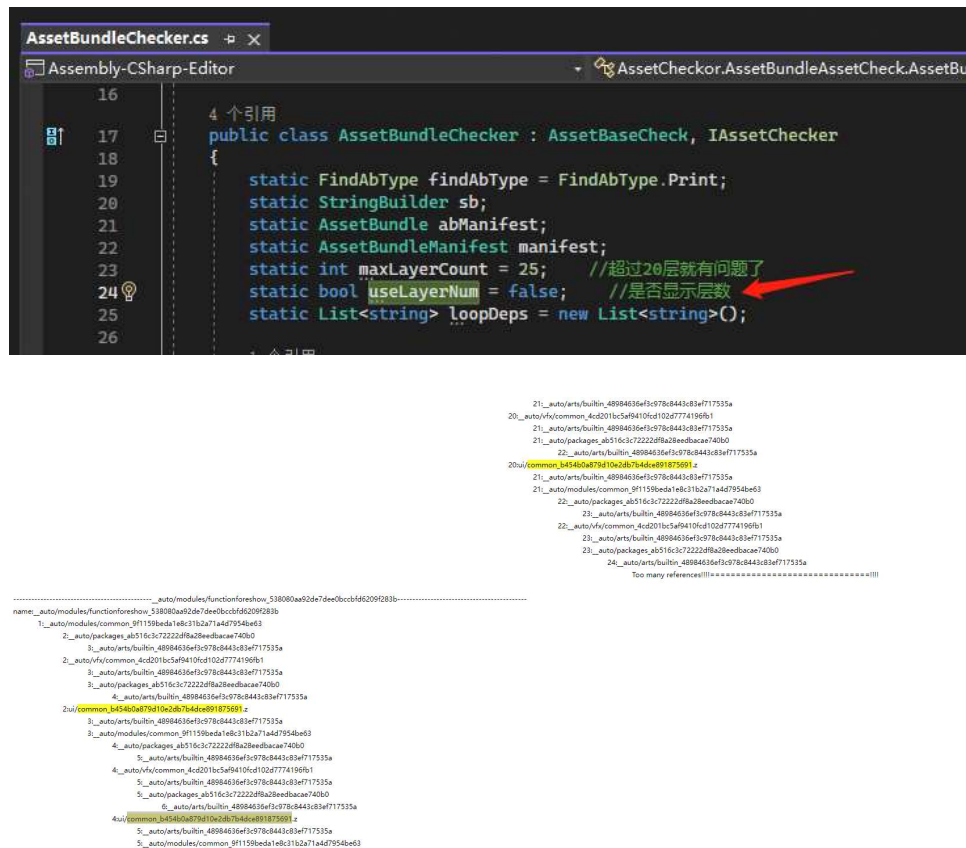
这个功能可以通过层级显示ab的加载依赖关系链，直到没有依赖，通过查看关系链可以检查ab的加载内存是否合理，为未来优化IO发热做好准备，在使用AB过程中容易出现循环依赖现象，是一直AB资源管理的畸形现象，通过这个功能也可以查看。



通过上图可以看到有一个ab：

ui/modules/alchemy_b79b8b9b41b6f86f32c7071316721c1d.z，它会不停的循环依赖下去，形成一个死循环，如果关系链累计超过25层，会自动断掉，并且打印出：Too many references!!!!=====!!!!

如果需要显示层数，需要修改下面的开关：



(3) 循环依赖，通过上面的可以发现AB依赖存在循环依赖，但是很困难找出循环依赖关系链，循环依赖功能可以打印完整的依赖列表，a依赖b，b依赖d，d依赖e，e依赖a，依赖层数不限都可查找出来。循环依赖有一个特点，就是以ab为单位的循环，其他依赖它的

ab，也会出现循环，但是只要解决这个根ab的循环，其他依赖它的ab都会解决掉。

